BIOINFO 203 · SPECIAL PROJECT

PharmaGenDSS

A Mini Clinical Genomics Decision Support System for Variant-Informed Prescribing

John Marquel Revilla · 2015-11080 · MEng IE (ASE)

Bioinfo 203 — Sem 2 AY 2025-2026 · 21 May 2026

The clinical problem

Two patients on the same drug and dose can have very different outcomes.

- Adverse drug reactions cause an estimated 6.5% of adult hospital admissions. [1]
- Much of this variation is inherited — the genes that process drugs differ from person to person. [2]
- Pharmacogenomic guidelines (CPIC and DPWG) now cover 100+ gene–drug pairs, but they are not used routinely at the bedside.
- Curated knowledge already exists — PharmGKB has the evidence, DrugBank has the drug records — but no single tool lets a prescriber ask, in one query, *given this patient's genetic profile, what should I do?*

Three real cases that motivate the system

Clopidogrel (a blood thinner)

Patients with certain CYP2C19 variants do not activate the drug well. The FDA recommends an alternative for these patients after a coronary procedure.

Warfarin (an anticoagulant)

Variants in CYP2C9 and VKORC1 can change the right dose by up to three-fold. CPIC's genotype-guided dosing has the highest evidence rating (Level 1A).

Abacavir (an HIV drug)

Patients with the immune marker HLA-B*57:01 are at risk of a severe reaction. Screening before prescription is now standard of care.

These are the kinds of decisions PharmaGenDSS surfaces, automatically, once a patient's genetic profile is on file.

Project objectives & scope

Primary objective

Build a small, web-accessible decision-support system that maps patient genetic variants to pharmacogenomic drug recommendations curated from PharmGKB and DrugBank, with transparent evidence provenance.

Specific objectives

- Ingest and normalize PharmGKB clinical/variant annotations and DrugBank drug records.
- Expose three driving query types: variant-driven, drug-driven, and gene-driven.
- Surface CPIC/DPWG evidence levels and citation provenance.
- Support role-based access for clinicians, pharmacists, curators, researchers.

In scope (proposal MVP)

- Variant–drug query API + minimal web UI.
- Curator dashboard with versioned ingest.
- Audit log + role-based access control.
- Single-tenant deployment via Docker Compose.

Out of scope (deferred)

- Direct EHR / FHIR Genomics integration.
- ML-based novel evidence ranking.
- Multi-tenant SaaS, billing, regulatory certification.

Stakeholders & intended users

Persona	Role	Primary goal	Key requirements
Prescribing clinician	Physician (internal med, oncology, cardiology)	Variant→drug lookup at point of care	Fast, ≤3-click lookup; clear recommendation; evidence levels.
Clinical pharmacist	Hospital/community pharmacist	Validate variant–drug pairs and dose adjustments	Full evidence, citation, dose-adjustment guidance.
Bioinformatics curator	PGx data steward	Ingest PharmGKB / DrugBank releases; manage versions	Versioned ETL, dry-run, rollback, audit.
Researcher	PGx / clinical genomics researcher	Aggregate queries by gene / drug class / population	Bulk export, anonymized audit trail, statistical filters.

Methodology & elicitation approach

Development methodology: Agile

- Requirements will evolve as clinicians give feedback — Agile is built around that.
- Aim for working software early: a small CYP2C19 demo can be shown in sprint 2.
- Containerized deployment and continuous integration so each build is reproducible.
- Spiral was considered and rejected — the risk profile is moderate, not the high-risk regime that makes spiral worth its overhead.

Elicitation techniques

- Focused ethnography — shadow clinicians during prescription rounds; observe what they actually consult.
- Stories & scenarios — structured: initial assumption → normal flow → what can go wrong → final state.
- Use cases — three driving interactions, written down formally.
- Document review — CPIC, DPWG, and the FDA biomarker list.

Validation — five checks at every iteration:

validity · consistency · completeness · realism · verifiability — applied before each sprint closes.

Driving scenario — clinician at point of care

Stage	Description
Initial assumption	A 58-year-old patient has just had a coronary procedure. The cardiologist needs to prescribe a blood thinner. The patient's CYP2C19 genotype is on file — a known slow-metabolizer profile for clopidogrel.
Normal flow	Clinician opens PharmaGenDSS and queries by patient ID. The system returns ranked drug options: clopidogrel flagged red (avoid for slow metabolizers, CPIC Level 1A); two alternatives flagged green. Clinician selects an alternative; system shows the supporting citation and exports a PDF for the chart.
What can go wrong	(a) The variant is not in PharmGKB — the system returns 'no evidence' explicitly, instead of a silent empty result. (b) CPIC and DPWG disagree — both recommendations are shown side-by-side; the system never picks silently. (c) The clinician role is not licensed for some DrugBank fields — the UI hides them and the redaction is logged.
Final state	A genotype-aware decision is reached in <30 s. An anonymized audit entry records the query, the recommendations shown, and the drug selected, for traceability.

User vs System requirements

User requirements (stakeholder language)	System requirements (developer-grade specification)
The clinician shall be able to look up drug recommendations using a patient's genetic profile.	FR4–FR6: variant-, drug-, and gene-driven query handlers, exposed as REST endpoints accepting standard variant and drug identifiers.
The pharmacist shall be able to inspect dose-adjustment guidance and the supporting evidence.	FR7: an evidence-provenance service that joins each recommendation to its CPIC or DPWG guideline and citation list.
The curator shall be able to refresh the data without disrupting live queries.	FR10: a versioned data-refresh pipeline that writes to a holding area first, then promotes atomically; supports rollback to the prior release tag.
The system should be easy to use and respond quickly.	Product NFRs: ≥99.5% availability; query under 2 s at p95; ≤3 clicks to first recommendation.
The system shall protect sensitive patient and licence-restricted data.	Organisational + External NFRs: TLS 1.3 in transit, AES-256 at rest, role-based access control; DrugBank academic-licence fields gated by user role.

Functional requirements — detailed (1/2)

Showing 4 of the 12 FRs (FR4, FR7, FR10, FR11) — the most consequential. Full set in the written report.

FR4 · Variant-driven query

Description:	Return ranked drug-variant interactions for one or more patient variants.
Inputs:	List of variant identifiers; optional drug-class or population filter.
Outputs:	Structured response listing each {drug, gene, variant, recommendation, evidence level, citations}.
Action:	System shall normalise the input, resolve the relevant gene, fetch annotations, rank by evidence level, redact fields by user role, and return.
Pre-condition:	Authenticated user; a current data release is loaded.
Post-condition:	Audit log entry written; response cached for 5 minutes.

FR10 · Curator data refresh

Description:	Run a versioned refresh of PharmGKB / DrugBank data, with dry-run and rollback.
Inputs:	Release tag (PharmGKB date, DrugBank version) and curator credentials.
Outputs:	Job report; the new promoted release tag; updated catalog entry.
Action:	System shall fetch the sources, validate the schema, write to a holding area, run integrity checks, and promote atomically; rollback if any check fails.
Pre-condition:	Authenticated curator; previous release tagged in the catalog.
Post-condition:	Catalog updated; audit log + downstream notifications dispatched.

Functional requirements — detailed (2/2)

FR7 · Evidence provenance

Description:	Show the originating CPIC / DPWG guideline and PubMed citations behind every recommendation.
Inputs:	ClinicalAnnotation ID.
Outputs:	Linked guideline metadata and a list of cited papers.
Action:	System shall traverse foreign keys to the source guideline, then look up citation details from a local cache or via a PubMed API call.
Pre-condition:	The annotation row exists; guideline metadata is pre-loaded.
Post-condition:	Provenance bundle returned; citation IDs cached.

FR11 · Audit log of queries

Description:	Every query writes one immutable audit row, with the user identity anonymised.
Inputs:	User token, query payload, timestamp.
Outputs:	AuditEntry persisted; export available for compliance review.
Action:	System shall anonymise the user ID using a daily-rotated key, then persist (timestamp, role, query hash, result summary, redactions).
Pre-condition:	Authenticated request.
Post-condition:	AuditEntry committed before HTTP 200 is returned to client.

Remaining FRs (drug-driven query, gene-driven query, filters, role-based access control, export) follow the same template — full set in the written report.

Non-functional requirements (Sommerville triad)

Product

- Performance: most queries answered in under 2 seconds.
- Reliability: at least 99.5% available during business hours.
- Usability: clinician finishes a lookup in 3 clicks or fewer.
- Security: industry-standard encryption in transit and at rest.
- Accuracy: cross-checked against the CPIC reference test set.

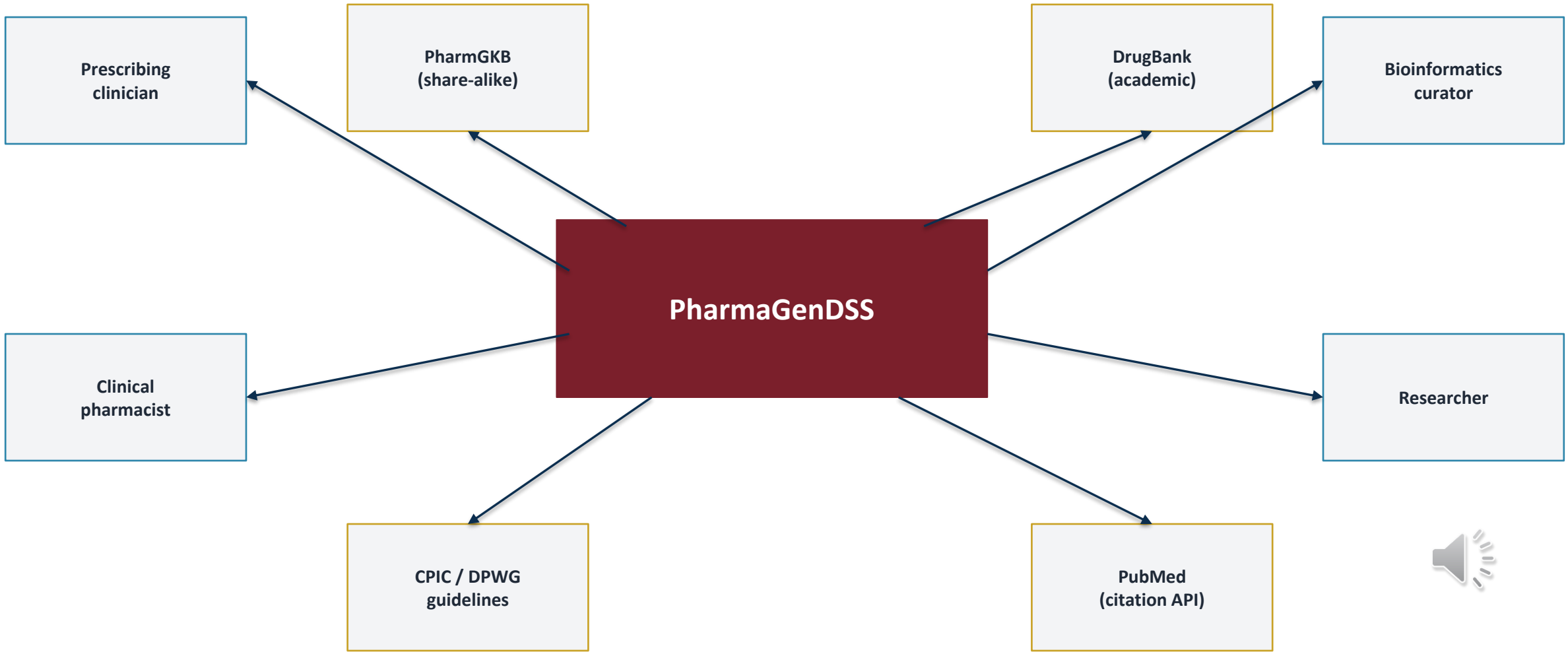
Organisational

- Process: standard version control and continuous integration.
- Standards: agreed conventions for naming variants and drugs.
- Documentation: requirements, system, technical, API, and user docs.
- Quality: automated tests cover at least 80% of the core code.
- Output: format compatible with the standard healthcare data exchange.

External

- Legal: respect the licences of PharmGKB (share-alike) and DrugBank (academic).
- Interoperability: standard web API; future hospital integration.
- Ethical: 'decision support — not diagnostic' disclaimer; minimal personal data collected.
- Regulatory-aware: aligned with the FDA biomarker reference list; no claim of being a medical device.

Contextual model — system context diagram



Blue = user/actor · Gold = external data source

Use cases (interaction model)

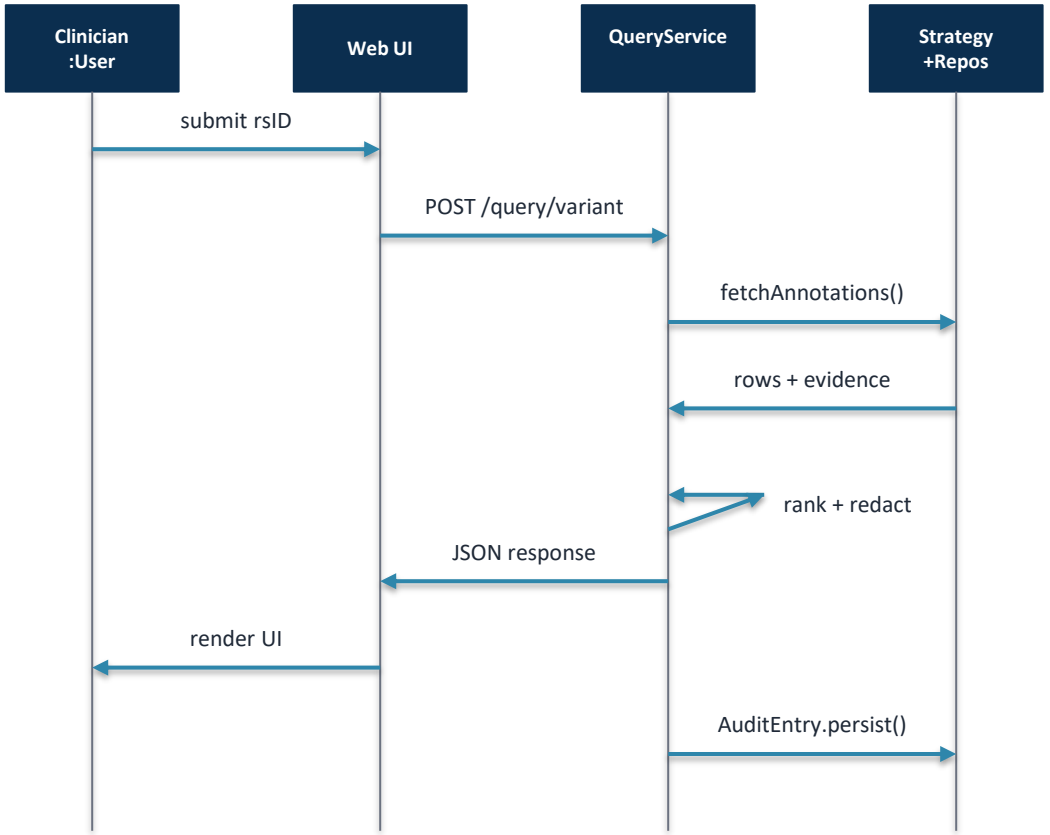
Field	UC1 · Look up drug recommendation	UC2 · Curate new release	UC3 · Aggregate cohort query
Actor	Prescribing clinician	Bioinformatics curator	Researcher
Description	Real-time variant-to-drug lookup at point of care.	Refresh PharmGKB / DrugBank data with dry-run + rollback.	Bulk query for genotype prevalence and drug-interaction frequency.
Data	Patient genetic profile; drug recommendations; evidence levels.	Source files; release tag; integrity-check report.	Cohort filters (gene, drug class, population); aggregate counts.
Stimulus	Clinician submits a query via web UI or API.	Curator triggers data refresh from the dashboard.	Researcher submits aggregate query with filters.
Response	System returns ranked recommendations with sources and licence redactions; logs the query.	System fetches, validates, stages, and promotes; emits notifications; updates the catalog.	System runs the aggregate query on a read-only replica; returns anonymised tabular results; logs.
Comment	Must respond in <2 s p95; never return silently when no evidence exists — explicitly say so.	Must be reversible — the holding-area schema is the safety net.	Cohort size ≥50 enforced to protect individual privacy.

Structural model — principal classes

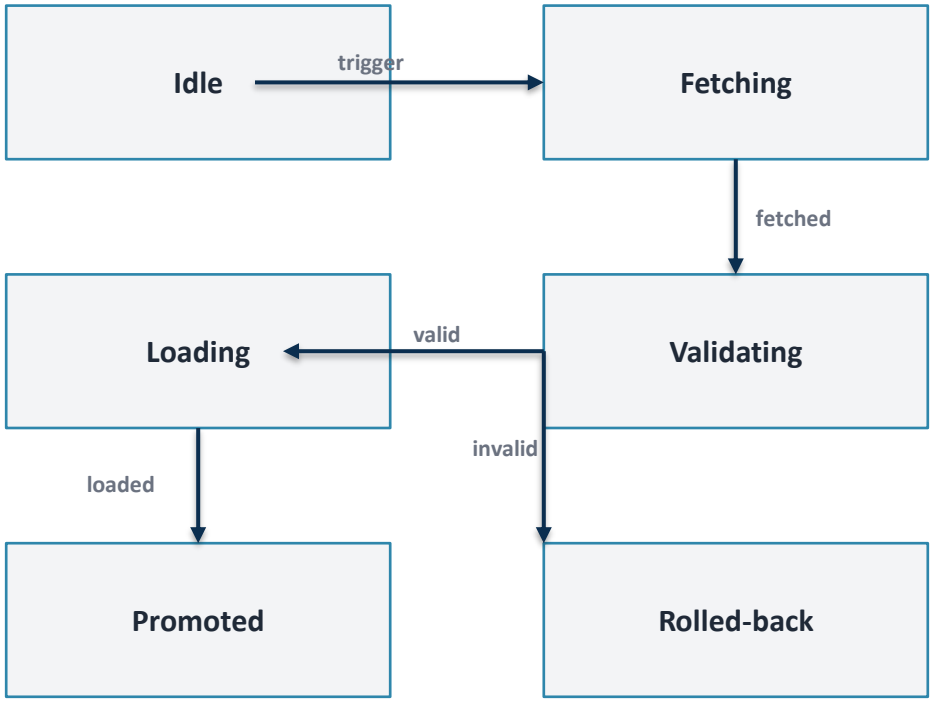
Class	Attributes	Operations
Variant	+ rsID: str + hgvs: str + gene_id: int + alleles: list[str]	+ normalize(): Variant + asKey(): str
Drug	+ drugbank_id: str + name: str + atc_code: str + rxnorm_id: str	+ asKey(): str + redactedView(role): dict
ClinicalAnnotation	+ ca_id: str + variant: Variant + drug: Drug + recommendation: str + evidence_level: str	+ provenance(): EvidenceRecord + rank(): int
EvidenceRecord	+ guideline: str (CPIC/DPWG) + citations: list[PMID] + release_tag: str	+ dereferenceCitations()
QueryService	- variantRepo, drugRepo, annoRepo - strategyMap: Strategy	+ byVariant(rsIDs) + byDrug(id) + byGene(symbol)
AuditEntry	+ user_hash: str + role: str + query_hash: str + ts: datetime	+ persist()

Behavioral model — sequence + state

Sequence: variant→drug query (UC1)



State: ETL ingestion job (UC2)



Architectural drivers & application architecture

Drivers (NFRs that shape architecture)

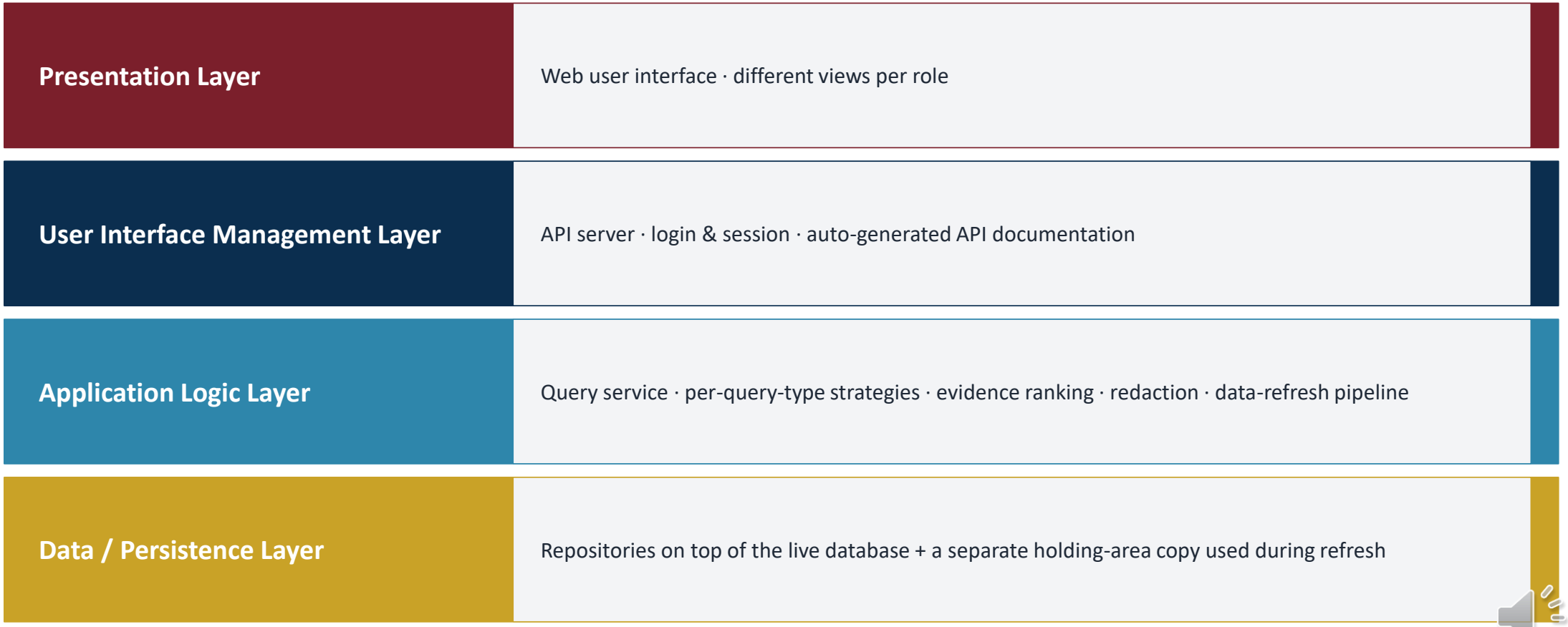
- Performance — answer queries quickly → in-memory caching, indexed data lookups.
- Security — protect data and respect licences → a guard layer that checks identity and role before any data is returned.
- Maintainability — monthly data refreshes → keep the data layer separable from the business logic.
- Reliability — refresh data without disrupting users → write to a holding area first, then swap atomically.
- Interoperability — work well with future systems → expose a well-documented standard web API.

Application architecture (Sommerville)

Information system / transaction-processing variant.

- User asks the system for information; the system reads or updates a database.
- Standard four-layer shape: user interface → user-comms → information retrieval → systems database (PS3 lineage).
- Inside the data-refresh path, we use a small pipe-and-filter pipeline as a sub-architecture.

Architectural pattern — Layered + Repository



The Repository pattern sits at the boundary with the database — each entity has its own dedicated lookup object, so the database engine could be swapped without touching the rest of the system.

Architectural views

Logical view

What the system is made of:
the main components and classes (query service, strategies, repositories, evidence service, audit service, data-refresh pipeline).

Process view

What runs at runtime:
api server processes, an asynchronous database connection pool, a separate worker for data refresh, and scheduled jobs.

Development view

How the code is organised:
/api · /core · /domain · /repository · /etl · /web · /tests · /infra (containers, CI).

Physical view

How it is deployed:
a set of containers — web, api, refresh worker, primary database with a read-only copy, and an in-memory cache.

Architecture ↔ NFR mapping

Each non-functional requirement maps to one architectural choice. Every choice is there for a reason.

Performance	Caching + indexed lookups → fast queries.
Security	Login at the API + redaction in the application logic → data is protected before it leaves the system.
Maintainability	Repository pattern + holding-area copy → monthly refreshes without disturbing live data.
Reliability	Pipe-and-filter pipeline with rollback → data refresh either fully succeeds or is fully reverted.
Interoperability	API-first design with auto-generated docs → other systems can integrate without bespoke contracts.

Tech stack & reuse strategy (hybrid)

Layer	Choice	Type	Why
Frontend	React + TypeScript + TanStack Query	OSS COTS (MIT)	Componentized; large pool of reusable charting/UI libs.
API framework	FastAPI 0.110+ (Python 3.12)	OSS COTS (MIT)	Auto OpenAPI; type-safe; async; Pydantic validation.
ORM / migrations	SQLAlchemy 2 + Alembic	OSS COTS (MIT)	Repository pattern fits naturally; mature migration tooling.
Database	PostgreSQL 16	OSS COTS (PostgreSQL Lic.)	JSONB for flexible annotation fields; mature; free.
Caching	Redis 7	OSS COTS (BSD)	Hot-read cache; rate limiting; TTL semantics.
Auth	OAuth2 password flow → JWT (PyJWT) + bcrypt	OSS COTS	Standard, audited, role claims for RBAC.
ETL / variant normalization	pandas, lxml, custom variant-name validator	Hybrid — OSS + custom	Standard data libs reused; pharmacogenomics-specific normalisation built in-house.
Container / IaC	Docker Compose, GitHub Actions	OSS COTS (Apache/MIT)	Host-target parity; CI on every push.
Evidence ranking	Custom scoring engine	Custom build	Specialised: weights CPIC and DPWG evidence levels; not a commodity.

OO design process & principal system objects

Sommerville's 5-stage OO design process

- 1. Define the context & modes of use → done (slides 5–7, 12).
- 2. Design the system architecture → done (slides 16–20).
- 3. Identify principal system objects → this slide.
- 4. Develop design models → structural + dynamic (next 2 slides).
- 5. Specify object interfaces → slide 22.

Principal system objects

- Domain entities: Variant · Gene · Drug · ClinicalAnnotation · EvidenceRecord.
- Service objects: a query service (Façade) · the data-refresh pipeline · evidence service · audit service.
- Per-query-type handlers (Strategy): one each for variant-, drug-, and gene-driven queries.
- Repository objects: one per entity, sitting between the application logic and the database.
- Cross-cutting helpers: login enforcement, role-based redaction, in-memory cache.

Object interfaces — plain-language sketch

Interface	What it offers (in plain terms)	Pre / Post conditions
Query Service (Façade)	Three look-up methods, one per query type: by variant, by drug, by gene. Each takes filters and a user context, returns a structured query result.	Pre: user is logged in; a current data release is loaded. Post: an audit entry is written; the result respects the user's role-based redactions.
Annotation Repository	Three find-by methods: by variant, by drug, by gene. Returns the matching clinical-annotation rows.	Indexed for fast lookup on the join keys.
Data-refresh Pipeline	Five small steps performed in order: 1. fetch — pull source files 2. validate — schema check 3. hold — write to a holding-area copy of the database 4. promote — atomic swap into the live data 5. rollback — abort and revert if anything fails	Each step is single-purpose and repeatable; rollback is exactly what makes the refresh safe.

Formal type signatures appear in the written report (TDD appendix).

Design patterns adopted

Pattern (category)	Where used	Why
Repository (architectural pattern)	Between the application and the database — one per entity.	The application doesn't talk to the database directly, so we can swap the storage engine without rewriting business logic.
Strategy (behavioural)	One handler each for variant, drug, and gene queries.	The three query types differ in how they look up and rank results. Strategy keeps those differences separate.
Façade (structural)	The query service is the single front door for all three look-up types.	Controllers (and any future client) only have to know one entry point.
Observer (behavioural)	The data-refresh pipeline announces when a release is promoted or rolled back; the audit service and cache listen.	Adding new listeners (for example, metrics) does not require changing the pipeline itself.
Factory Method (creational)	An evidence-record factory produces guideline-specific subclasses (CPIC, DPWG, …).	Each guideline source has a different provenance shape; the factory hides that detail from callers.
Singleton (creational, sparingly)	Used only for cross-cutting helpers like the cache and the login middleware.	These exist exactly once per process; there is no reason to have multiple instances.

Implementation issues + Open-source development

Reuse

- OSS COTS for commodity layers (FastAPI, SQLAlchemy, PostgreSQL, React).
- Application-system reuse considered for general LIMS — rejected: the gating and ranking logic is too domain-specific to be off-the-shelf.
- Curated PharmGKB & DrugBank datasets reused; not regenerated.

Configuration management

- Git/GitHub flow; semantic versioning on tags.
- GitHub Actions: pytest, ruff, mypy on every push.
- Data-release tags tracked alongside code tags in the catalog table.
- Schema migrations are tooled; never edit the database by hand.

Host-target development

- Dev: Windows 11 / macOS · Linux Docker daemon.
- Target: Linux container in single-tenant deployment.
- Docker Compose pins versions of OS, Python, Postgres, Node.
- Avoids "works on my machine" via target-parity images.

Open-source

- Project licence: MIT (permissive).
- Derived PharmGKB data: share-alike obligation preserved.
- DrugBank fields: academic licence honoured via role-based redaction.
- Dependencies scanned for known vulnerabilities on every build.

Risks · mitigations · documentation plan

Risks & mitigations

Risk	Mitigation
DrugBank academic-licence ambiguity	Role-based redaction at API boundary; legal review pre-deployment.
Identifier mismatch between PharmGKB and DrugBank	Drug-name bridge using a public terminology service + curator-flagged unresolved set.
Variant-naming differences across sources	Normalisation layer with a name-format validator and a golden-set of test cases.
Stale data between releases	Scheduled monthly refresh + visible 'last refresh' badge in UI.
Scope creep into hospital EHR integration	Explicitly deferred; documented as Phase 4.

Documentation plan (Session 6 + PS4)

- Product requirements doc · system architecture doc · technical design doc.
- Source-code documentation generated from the code.
- API documentation generated automatically from the API server.
- End-user manual + administrator runbook.

Process docs

- Agile product roadmap + sprint backlogs.
- Coding standards + test strategy.
- Release checklist + working papers.

Roadmap & summary

Phase 0 · Proposal	Phase 1 · MVP	Phase 2 · Full DSS	Phase 3 · Hardening	Phase 4 · Future
Reqs · models · ER · stack	ETL + read-only API + minimal UI	Auth · audit · all query types	Dashboards · exports · CPIC sync	FHIR Genomics · ML evidence rank

Summary

- PharmaGenDSS proposes a tight-scope, transaction-processing information system that maps a patient's genetic profile to drug recommendations curated from PharmGKB and DrugBank.
- Layered + Repository pattern with a pipe-and-filter sub-architecture for data refresh; deployed via Docker Compose; Agile process with a DevOps complement.
- Design follows Sommerville's frameworks (four system perspectives, 4+1 views, classic design patterns, Session 5 implementation issues) and the conventions established across Problem Sets 1–4.
- Open-source-first stack, with custom build only for the domain-specific ranking and variant-name normalisation logic.

References

- [1] Sommerville, I. Software Engineering, 10th ed. Pearson, 2014.
- [2] Pirmohamed, M. et al. "Adverse drug reactions as cause of admission to hospital: prospective analysis of 18 820 patients." BMJ 329(7456): 15–19, 2004. doi:10.1136/bmj.329.7456.15
- [3] Whirl-Carrillo, M. et al. "An Evidence-Based Framework for Evaluating Pharmacogenomics Knowledge for Personalized Medicine." Clin. Pharmacol. Ther. 110(3): 563–572, 2021. (PharmGKB)
- [4] Wishart, D. S. et al. "DrugBank 5.0: a major update to the DrugBank database for 2018." Nucleic Acids Res. 46(D1), 2018.
- [5] Relling, M. V., Klein, T. E. "CPIC: Clinical Pharmacogenetics Implementation Consortium of the Pharmacogenomics Research Network." Clin. Pharmacol. Ther. 89(3): 464–467, 2011.
- [6] Swen, J. J. et al. "Pharmacogenetics: from bench to byte — an update of guidelines." Clin. Pharmacol. Ther. 89(5), 2011. (DPWG)
- [7] U.S. FDA. Table of Pharmacogenomic Biomarkers in Drug Labeling. fda.gov/drugs/science-and-research-drugs/table-pharmacogenomic-biomarkers-drug-labeling
- [8] HL7 International. FHIR Genomics Implementation Guide. hl7.org/fhir/genomics.html
- [9] den Dunnen, J. T. et al. "HGVS Recommendations for the Description of Sequence Variants: 2016 Update." Hum. Mutat. 37(6): 564–569.
- [10] Liu, S. et al. "FastAPI: A Modern, Fast (high-performance) Web Framework for Building APIs." tiangolo.com/fastapi (project docs).
- [11] Bayer, M. SQLAlchemy 2.0 Documentation. sqlalchemy.org.
- [12] Bioinfo 203 Lecture Slides — Sessions 1–6, AY 2025–2026.

Thank you.